

Lecture 2 RISC-V Introduction

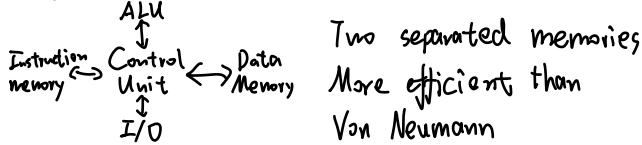
1. Instruction set architecture (ISA)

(1) Types of architecture

(i) Von Neumann Architecture

Memory unit only one, in CPU

(ii) Harvard Architecture



(2) Instructions:

(i) Simplicity favors regularity

(ii) Smaller is faster

register: 32

(iii) make common case fast

Immediate

$x_0 = 0$

(iv) RISC-V Registers

Register	Alternative name	Description
x0	zero	the constant value 0
x1	ra	Return address
x2	sp	Stack pointer
x3	gp	Global pointer
x4	tp	Thread pointer
x5 - x7	t0 - t2	Temporaries
x8	s0/fp	Saved register/Frame pointer
x9	s1	Saved register
x10-17	a0-7	Function arguments/Return values
x18-27	s2-11	Saved registers
x28-31	t3-6	Temporaries

(v) Immediate & Sign Extension

An I-type can have 12 bits of immediate.

upper bit are the same as the top bit.

(3) Data Transfer Operations

Arithmetic operations can only be performed on registers $\times$ Branching far away

Only memory actions are loads & stores.

Register 100-500 times faster

(i) Memory addresses are in bytes

words take on the address of their LSB

RISC-V is Little Endian

(ii) Logic operations

slli by i bit = $\times 2^i$

srli by i bit = $\div 2^i$ (only unsigned)

$\times$ not operation $\Rightarrow$ xor x1, x2, -1

Lecture 3 RISC-V Instruction Format

1. Pseudo-instructions

bgt x2, x3, foo $\rightarrow$ blt x3, x2, foo

lw, mv, li, not...

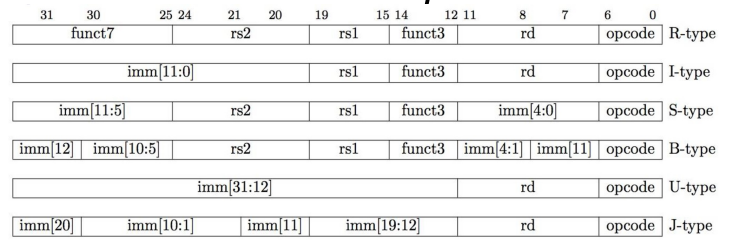
$\times$ Basic blocks (no jump from/to inside)

No embedded branches (at the end)

No branch target (at the beginning)

Can be optimized by compiler.

2. Instructions in Binary



32 bit instructions

- R: register-register operations
- I: register-immediate ALU operations and loads
- S: Stores
- B: Branches
- U: 20-bit upper immediate instructions
- J: jumps

R format: `add x18, x19, x10`

00000000	01010	10011	000	10010	0110011
ADD	rs2=10	rs1=19	ADD	rd=18	Reg-Reg OP

I format: imm[11:0] can hold [-2048 ~ 2047]

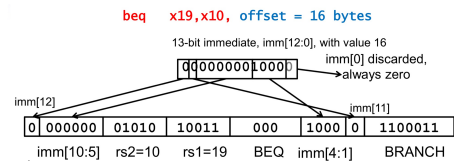
Load with I-type imm $\Rightarrow$ offset from rs1 address

S format:

imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
offset[11:5]	src	base	width	offset[4:0]	STORE

12 bit offset

B format:



± 12 byte offset, PC always point to instruction

32 bit instructions are "word-aligned"

address is always a multiple of 4.

replace with jump

U format:

imm[31:12]	rd	opcode
U-immediate[31:12]	dest	LUI
U-immediate[31:12]	dest	AUIPC

LUI: Load Upper Imm, $rd = imm \ll 12$

AUIPC: Add Upper Imm to PC, $rd = PC + (imm \ll 12)$

J format:

imm[20]	imm[10:1]	imm[11]	imm[19:12]	rd	opcode
1	10	1	8	5	7
offset[20:1]				dest	JAL

j al saves PC+4 in rd (next ins)

set $PC = PC + offset$

Long jumps $\Rightarrow$ jalr rd, rs, imm (i format)

lui, jalr

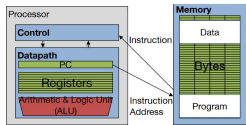
Lecture 4 Procedure Calling

1. Steps

- (1) Put parameters in a place where procedure →
 - (2) Transfer control
 - (3) Acquire the storage resources
 - (4) Perform the task
 - (5) Put result in a place where caller →
 - (6) Return to some point
- ✗ Program Execution

PC (program counter) is special internal register inside processor holding byte address of next instruction to be executed

Instruction is fetched from memory, then control unit executes instruction using datapath and memory system, and updates program counter (default is add +4 bytes to PC, to move to next sequential instruction)



2. RISC-V registers and convention

- a0-a7 for function arguments
- a0-a1 for return values
- ra for return address
- if move, use stack

3. How to Transfer control

jal ra, target j label
:
jr register
jalr zero, 0(ra)

4. Where are the local variables stored

sp stack pointer last used place at the stack
Automatic data, stack; Programs, text
Dynamic data, heap; Constants, Static data
caller-saved

- The function invoked (the callee) can do whatever it wants to them!
- Means that the caller can not count on their contents not being destroyed

callee-saved

- The function invoked must restore them before returning (if used)

5. Procedures

(1) Leaf Procedure: calls nothing

↳ Save and restore

may use to, t1...
Everytime store: addi sp, sp, -4n adjust stack for n items
...
sw s1, 4(sp)
sw s0, 0(sp)
end: lw s0, 0(sp)
lw s1, 4(sp)
addi sp, sp, 4n
jr ra

(2) Non-Leaf Procedure: calls other procedures

For nested call, (may be recursion)

caller needs to save on the stack

```

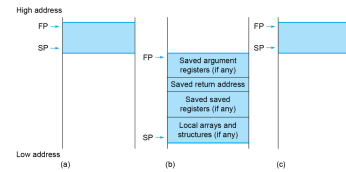
int sumSquare(int x, int y) {
    return mult(x,x)+y;
}
a0, a1

sumSquare:
1 addi sp, sp, -8 # reserve space on stack
2 sw ra, 4(sp) # save ret addr
3 sw a1, 0(sp) # save y
4 mv a1, a0 # prepare 2nd argument for mult
5 jal mult # call mult
6 lw a1, 0(sp) # restore y
7 add a0, a0, a1 # mult() + y
8 lw ra, 4(sp) # get ret addr
9 addi sp, sp, 8 # restore stack
10 jr ra
mult:

```

✗ Callers save registers before call

Callees must save registers before use

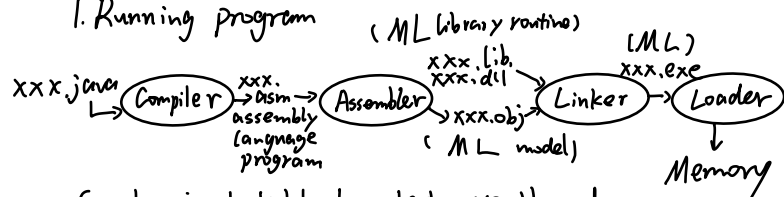


FP: frame pointer
栈底 or ± offset

Functions called with jal, ret with jalr

Lecture 5 Performance

1. Running program



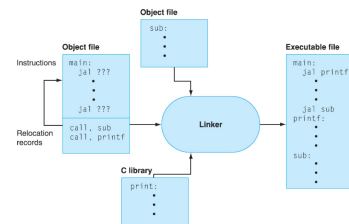
Compiler: input high level, output assembly code

Assembler: output object code

read and use directives .align, word...

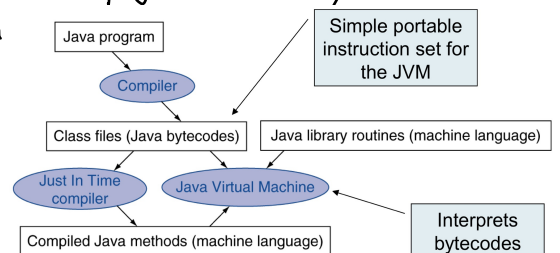
replace pseudo-instruction

Linker: output executable code



Loader: load program to memory

✗ Java



2. Measuring Execution Time

(1) CPU clocking:

period (time) 1s = 1000ms = 10⁶μs = 10⁹ns = 10¹²ps

frequency (rate) 1GHz = 1000MHz = 10⁶kHz = 10⁹Hz

(2) CPU Time = # of Clock cycles × Clock Period
= # of Clock cycles / clock rate

from ISA

(3) Instruction count and CPI

$\text{Clock cycles} = \text{Instruction count} \times \text{CPI}$
 $\text{CPU Time} = \text{Ins count} \times \text{CPI} \times \text{period}$
 $= \frac{\text{Ins count} \times \text{CPI}}{\text{Clock rate}}$

(4) If different instruction takes different cycles

$\text{clock cycles} = \sum_{i=1}^n (\text{CPI}_i \times \text{Ins count}_i)$

$\text{weighted average CPI} = \frac{\sum_{i=1}^n (\text{CPI}_i \times \text{Ins count}_i)}{\text{total Ins count}}$

$\Rightarrow \text{CPU Time} = \text{IC} \times \text{CPI} \times T_c$
 $= \frac{\text{IC} \times \text{CPI}}{f}$

3. Energy Consumption

(1) Def:

Consumption = dynamic energy + static energy
 Dynamic (transistors 0 $\Rightarrow$ 1) is primary
 Static when no transistor changes

(2) Types:

$0 \rightarrow 1 \rightarrow 0 \quad E \propto \text{Capacitive load} \times \text{Voltage}^2$
 $0 \rightarrow 1 \text{ or } 1 \rightarrow 0 \quad E \propto \frac{1}{2} \times \text{Capacitive load} \times \text{Voltage}^2$
 per second (power) $P \propto \frac{1}{2} \times \text{Capacitive load} \times \text{Voltage}^2 \times \text{Frequency switched}$

* Power trend: growing density of transistors, multi-core, slowing rate of performance improvement, power/thermal constraints

4. SPEC CPU Benchmark

Server side java operations per second

power: Watt

Load: percentage 0% 10% ... 100%

$\text{overall ssj_ops per watt} = \left(\sum_{i=1}^n \text{ssj_ops}_i \right) / \left(\sum_{i=1}^n \text{power}_i \right)$

* Design processors to make power proportional to the load

5. Amdahl's Law

Design about bottle-neck make common case fast

$\text{Speedup} = \frac{1}{(1-f) + \frac{f}{S}}$
 (1-f) 其他不并行
 f 并行比例
 S 并行倍数

Lecture 6 Integers Arithmetic

1. Addition

signed, unsigned * carries in each bit

746: $\begin{array}{r} 000111 \\ 000110 \\ \hline 001101 \end{array}$

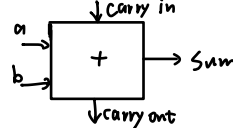
Overflow: result out of range

(i) $0 + 0$ no overflow

(ii) $0 + 0 = 0$
 (iii) $0 + 0 = 0$ } overflow

Prod [0]
 1: Prod left half + M_{and}
 Shift right prod
 0: shift right prod
 If iter = #bit stop

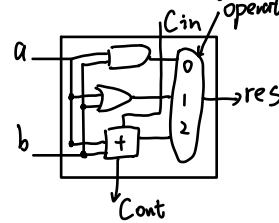
2. 1 bit adder



$\text{sum} = (a \oplus b \oplus \text{cin}) + (a \& b \& \text{cin}) + (a \& b) + (a \& \text{cin}) + (b \& \text{cin})$
 $\text{Cout} = (b \& \text{cin}) + (a \& \text{cin}) + (a \& b)$

3. 1 bit ALU and 32 bit ALU

(1) 1 bit Arithmetic logical unit

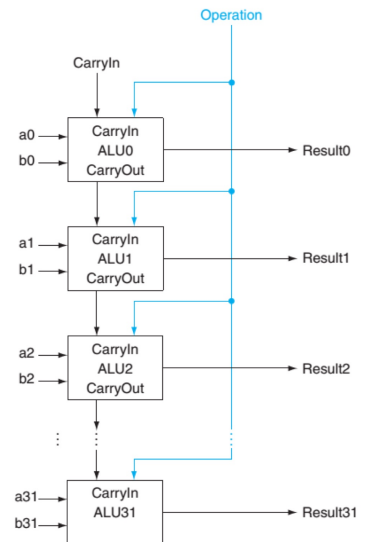


(2) 32 bit ALU

Normally, first cin = 0

Case (op)

0: &, and 1: | or 2: + add



4. Integer subtraction

Add negation of 2nd operand

$\begin{array}{r} +7 \quad 0000111 \\ -6 \quad 1111101 \\ \hline 1 \quad 0000001 \end{array}$

Overflow

(i) no $0 - 0$ or $1 - 1$

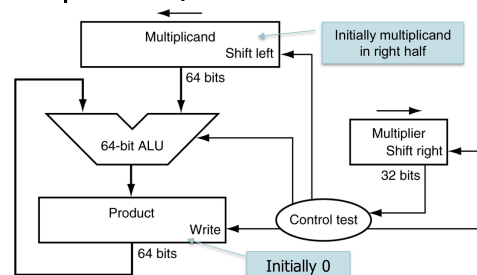
(ii) yes $0 - 1 \rightarrow 1$

(iii) yes $1 - 0 \rightarrow 0$

* Saturating operation: On overflow, result is largest value

5. Multiplication

(1) Multiplier (original)



Iter	Step	Multiplier	Multiplicand	Product
0	init	1001	0001000	00000000

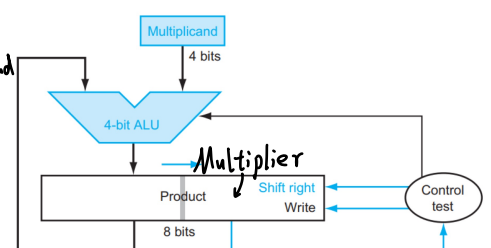
1 $1 \Rightarrow \text{Prod} = \text{Prod} + \text{M}_{\text{and}}$

Shift M_{and}, Shift M_{er} 0100 00010000 0001000

2 0 $\Rightarrow$ no op

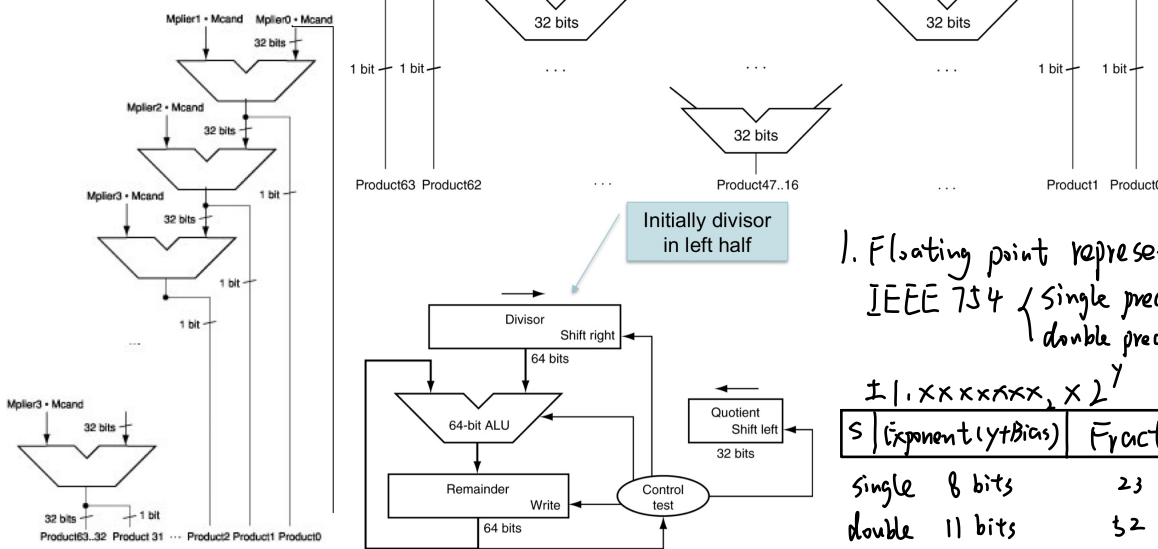
Shift M_{and}, Shift M_{er} 0010
 if it is the number of original bit:
 return (multiplier = 0)

(2) Optimized



Lecture 7 Floating point Arithmetic

(3) Faster Multiplier



6. Division

(1) Ordinary hardware

(i) check for 0 divisor

(ii) Long division approach:

If divisor > dividend bits: 1 in quotient, subtract; Or, 0 bit in quotient

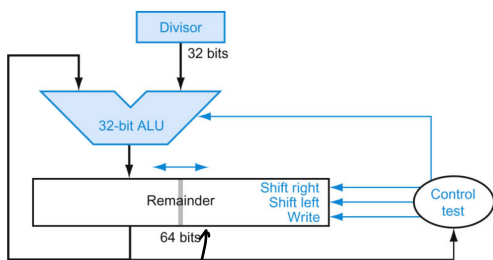
(iii) subtract, if remainder < 0, add back

(iv) divide using nbs; adjust sign after finish

$$Rem = Rem - Div$$

Rem < 0, +Div, shift 0 to Q, shift Div →
Rem ≥ 0, shift 1 to Q, shift Div →
(#bit + 1 iterations)

(2) Optimized Divider



Init: dividend and shift left 1 bit

1: sub divisor from left remainder

2: if < 0, restore, shift remainder left 1 bit, goto 4

3: shift remainder left 1 bit, new rightmost 1

4: Repeat 1 to 3 # of bit times

5: shift the left half of remainder right 1 bit

then Left is remainder, right is quotient

(3) Signed division

convert to positive and adjust sign later

被除数与除数同号

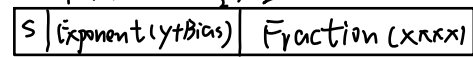
如果被除数与除数不同号,商为负

$$Dividend = Divisor \times Quotient + Remainder$$

1. Floating point representation

IEEE 754 { single precision
double precision

$$\pm 1.xxxxxxx \times 2^y$$



single 8 bits

23 bits

double 11 bits

52 bits

$$X = (-1)^S \times (1 + \text{Frac}) \times 2^{(\text{Exp} - \text{Bias})}$$

Normalize significand $1.0 \leq |\text{significand}| < 2.0$

Exp = actual exp + Bias

Exp is unsigned single: 127, double: 1023

* Decimal to FP

$$\text{e.g. } 24.5 \rightarrow \frac{11000}{24} \cdot \frac{1}{0.5} = (-1)^0 \times 1.10001 \times 2^4$$

$$E_{\text{bias}} = 4 + 127 = 131 = 10000011_2$$

$$F_{\text{rac}} = 10001 \Rightarrow 0.10000011 - 100010000000000000000000 \Rightarrow \text{hex}$$

* FP to decimal

$$1.1000001100000000, S=1, E_{\text{xp}}=10000011=129 \rightarrow \text{true } E_{\text{xp}}=2$$

$$\text{Fraction} = 01000000 \Rightarrow (-1)^1 \times (1 + 0.01)_2 \times 2^2 = (-1)^1 \times 1.25_{10} \times 2$$

$$2. \text{Range} = -5.0_{10}$$

(1) Exponent (all 0) and (all 1) are reserved

Smallest:

$$E_{\text{xp}} = 00000001 \Rightarrow \text{actual is } 1 - \text{bias} = \dots$$

$$F_{\text{rac}} = 00000000 \Rightarrow \text{significand} = 1.0$$

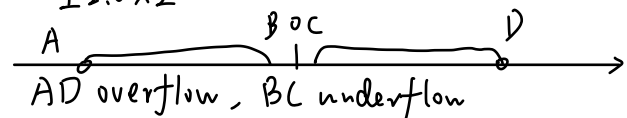
$$\Rightarrow \pm 1.0 \times 2^{-1-\text{bias}}$$

(2) Largest:

$$E_{\text{xp}} = 11111110 \Rightarrow \text{actual} = E_{\text{xp}} - \text{bias}$$

$$F_{\text{rac}} = 11111111 \Rightarrow \text{significand} \approx 2.0$$

$$\pm 2.0 \times 2^{E_{\text{xp}} - \text{bias}}$$



AD overflow, BC underflow

Single precision		Double precision		Object represented
Exponent	Fraction	Exponent	Fraction	
0	0	0	0	0
0	Nonzero	0	Nonzero	± denormalized number
1-254	Anything	1-2046	Anything	± floating-point number
255	0	2047	0	± infinity
255	Nonzero	2047	Nonzero	NaN (Not a Number)

±∞: divided by 0

NaN: 0/0, ∞-∞

$$\text{* Denormalized number } 0.00001 \times 2^{-126}$$

* Infinity and Nan.

Inf $\Rightarrow \text{Exp} = 111 \dots 111$, Fraction = 000...000

Nan $\Rightarrow \text{Exp} = 111 \dots 111$, Fraction $\neq 000 \dots 000$

illegal action, undefined result

(3) Precision (relative)

某FP A 的相对误差 $\Delta A / |A|$

ΔA 为最小可分辨率误差, 最后一位 fraction bit

$$\Rightarrow \frac{\Delta A}{|A|} = \frac{2^{-23} \times 2^y}{1.000 \dots \times 2^y} = \frac{2^{-23}}{1.000 \dots} \approx 2^{-23}$$

Decimal precision bit $\times (\log_{10} 2)^{\text{single}} \Rightarrow 23 \times 0.3 \approx 6$ digits

23 bit (single) $2^{-23} \times 2^y$

3. Addition

(1) Step: ① Align binary points

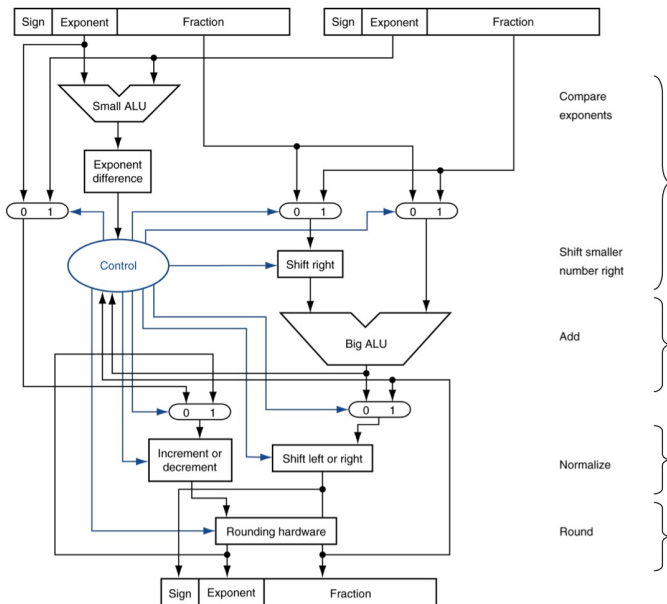
(shift number with smaller exp)

② Add significand

③ Normalize result, check for over/underflow

④ Round and renormalize if needed

(2) Adder



4. Multiplication

(1) Step: ① Add exponent

$$\text{unbiased } (\text{exp}_1 + \text{exp}_2) = \text{exp}_{\text{new}}$$

$$\text{biased } (\text{exp}_1 + \text{exp}_2) - \text{bias} = \text{exp}_{\text{new}}$$

② Multiply significands

③ } same with add

④

⑤ Determine sign

(2) Multiplier: same with adder, replace ALU

5. Accurate Arithmetic

Extra bit for precision (guard, round, sticky)

舍入: round up, round down, Truncate, round to nearest even

4舍6入5取偶

6. Matrix multiplication

```

li x28, 32 # x28 = 32 (row size/loop end)
li x5, 0 # i = 0; initialize 1st for loop
L1: li x6, 0 # j = 0; initialize 2nd for loop
L2: li x7, 0 # k = 0; initialize 3rd for loop
    slli x30, x5, 5 # x30 = i * 2^5 (size of row of c)
    add x30, x30, x6 # x30 = i * size(row) + j
    slli x30, x30, 3 # x30 = byte offset of [i][j]
    add x30, x10, x30 # x30 = byte address of c[i][j]
    fld f0, 0(x30) # f0 = c[i][j]
L3: slli x29, x7, 5 # x29 = k * 2^5 (size of row of b)
    add x29, x29, x6 # x29 = k * size(row) + j
    slli x29, x29, 3 # x29 = byte offset of [k][j]
    add x29, x12, x29 # x29 = byte address of b[k][j]
    fld f1, 0(x29) # f1 = b[k][j]
    slli x29, x5, 5 # x29 = i * 2^5 (size of row of a)
    add x29, x29, x7 # x29 = i * size(row) + k
    slli x29, x29, 3 # x29 = byte offset of [i][k]
    add x29, x11, x29 # x29 = byte address of a[i][k]
    fld f2, 0(x29) # f2 = a[i][k]
    fmul.d f1, f2, f1 # f1 = a[i][k] * b[k][j]
    fadd.d f0, f0, f1 # f0 = c[i][j] + a[i][k] * b[k][j]
    addi x7, x7, 1 # k = k + 1
    bltu x7, x28, L3 # if (k < 32) go to L3
    fsd f0, 0(x30) # c[i][j] = f0
    addi x6, x6, 1 # j = j + 1
    bltu x6, x28, L2 # if (j < 32) go to L2
    addi x5, x5, 1 # i = i + 1
    bltu x5, x28, L1 # if (i < 32) go to L1
    
```

7. Parallelism

(1) Subword parallelism

2 x 64 bit add

128 bit adder $\Rightarrow$ 16 x 8 bit add 4 x 32 bit add

Data level parallelism

(2) SIMD 单指令, 多数据 (重复加速)

128 bit 同时 4个 32 bit

Lecture 8 The Processor

1. Process

(1) Analyze instruction set

(2) Select set of datapath components

establish clocking methodology

(3) Assemble datapath

(4) Assemble the control logic

Memory to store instructions, data

Registers, ALU, control logic

Use PC to pull instructions out of cache

read registers

Building units

Combinational elements

Gates, Adder, Mux, ALU...

Sequential elements

* control bit

Instruction Memory and PC

Instruction Memory

input: 32 bit address, output: 32 bit instruction

PC

input, output: 32 bit address of instruction

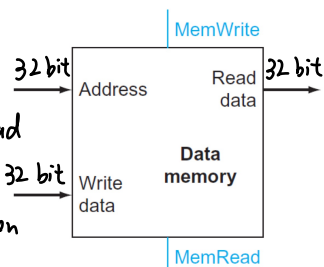
• Data memory

Read enable = 1

address select mem to be read

Write enable = 1

address select mem to be written



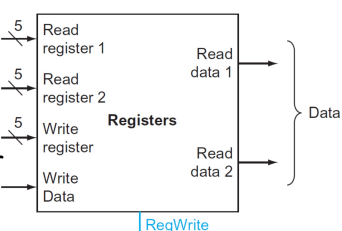
• Registers (32 registers)

3 register numbers 5 bit

register read is about

32 regs and 2 32:1 Mux

register write is about



1 $n \rightarrow 2^n$ decoder, reg data, write enable

• Other shift left, imm-gen

• Clocking Methodology

State element 1 $\rightarrow$ (Combinator) $\rightarrow$ (SE2)

Clock time > time of signal SE1 $\rightarrow$ SE2

SE can be read and write in the same clock (long enough)

PC, memory, regs all need clocking

2. Instruction

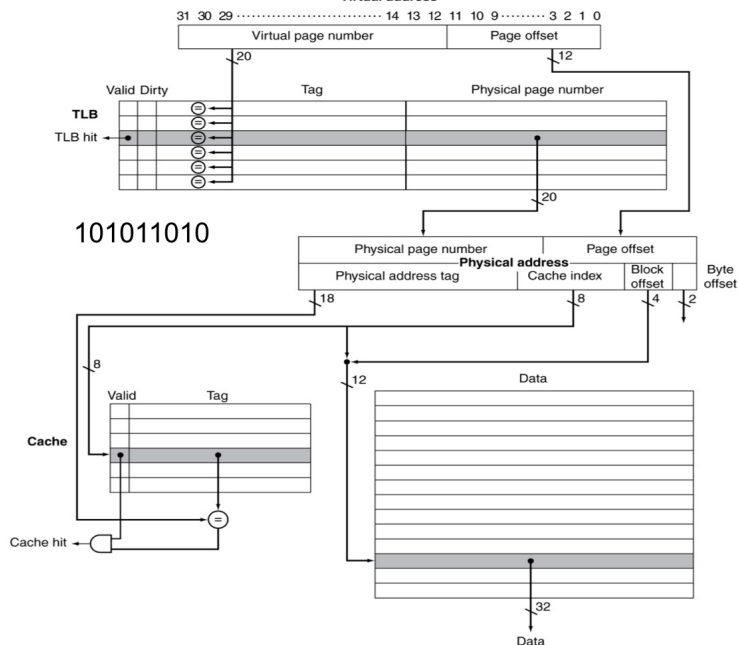
(1) Fetch: PC $\rightarrow$ ins, PCt = 4

(2) ALU control

Aluop = 00 Load/store: add

01 Branch: sub

10 R-type



0001 0010 0011 0100 0101 0110 0111 1000 1001 1010 1011 1100

Bit position	1	2	3	4	5	6	7	8	9	10	11	12
	0	1	1	1	0	0	1	0	1	0	1	0
Encoded data bits	p1	p2	d1	p4	d2	d3	d4	p8	d5	d6	d7	d8
Parity bit coverate	p1	X		X		X		X		X		X
	p2		X	X		X	X			X	X	
	p4				X	X	X	X				X
	p8							X	X	X	X	X

$$Vaddr = VPN + Pageoffset$$

↓ TLB or PTE ↓ copy

$$Paddr = PPN + Pageoffset$$

TLB: translation Look-aside Buffer

Lecture 9 Pipeline

9.1 Pipeline speedup

1. Performance issue

Longest delay determines clock period

Critical: Load ins, IF → ID → EXE → MEM → WB

$$T_{ins} = \begin{cases} T_c & \text{single cycle} \\ \sum T_c & \text{pipeline} \end{cases} \Rightarrow \text{increase throughput} \quad \text{Latency } \times$$

9.2 Hazards

1. Structure

Ins and Data use Mem simultaneously

⇒ 分开两块 Mem

2. Data

Use-use / Load-Use

上一个未写回 还未load出来

⇒ ① Stall: add NOP

use-use: 2 stall ID else 3 stall

load-use: 2 stall

② Forwarding

(use-use) EX-EX / MEM-EX (Load-use)

EX MEM EX
E_y E_x

③ stall + forwarding (load-use)

still need 1 stall even if forwarding

④ Code schedule

avoid use load result in the next ins

3. Control

branch/jump has addr.next at EX, but next ins begin

⇒ ① Stall on branch

② Prediction: ✓ no stall, ✗ redo

(i) static: software: ② 2k mostly ✓; 前 2k mostly ✗

(ii) dynamic: hardware: from previous record

Lecture 10 Instruction Level Parallelism

10.1 Multiple issue $CPI = \frac{1}{ZPC}$

1. Static multiple issue (software compiler)

① VLIW: ALU/branch + Load/store

Loop: ... cycle 1

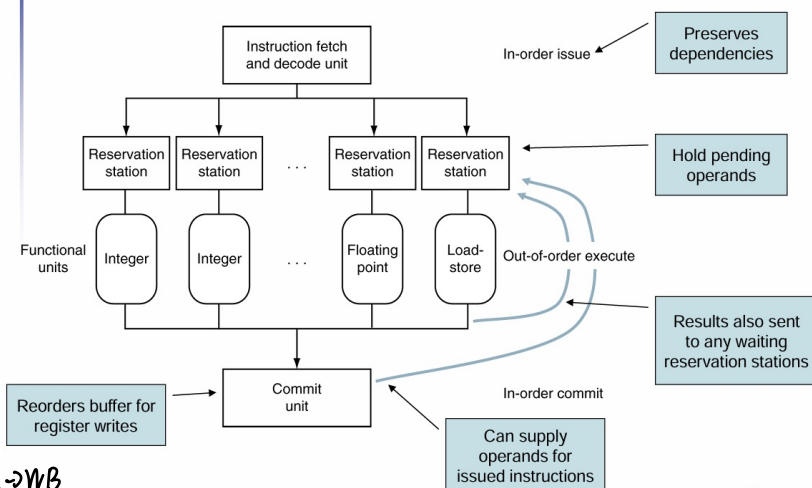
② Loop unrolling

use more reg to reduce stall/wait

2. Dynamic

Out of order exe

① Super scalar: In order commit



② speculation

(i) on branch ✓ do ✗ unroll, delay

(ii) on Load

Lecture 11 Memory Hierarchy I

11.1 Storage technology

1. Categories: SRAM, DRAM, Flash, Magnetic disk

2. Hierarchy: Reg → Cache → Main Mem → Secondary Storage
Larger, cheaper, slower, denser

3. 90/10 rules: 10% code executed 90% of time

Locality: Temporal: 被使用的很快又被使用

Spatial: 被使用的附近很快被使用

4. Terminology:

Block (line): unit of copying

Hit: data appear in upper level

Hit rate: $\frac{\text{hit}}{\text{access}}$, Hit time: time to access

Miss: data needs to be retrieved from lower level

Miss rate = $\frac{\text{miss}}{\text{access}}$ = 1-hit rate, miss penalty: time to fetch block

Hit Time << Miss Penalty

11.2 Cache Memory

1. Directed Mapped Cache

Location determined by address

One data in memory is mapped to only 1 location in cache

$$\text{Block addr} = \lfloor \text{byte addr} / \text{block size} \rfloor$$

index location = (Block addr) % (#Blocks in cache)

Index | V | Tag | Data (每个Block内容)

#Blocks = #Data / block size

#Byte offset = $\log_2(\text{block size})$ = (word + byte) offset

Index bit = $\log_2(\# \text{Blocks})$

Tag = Total - Index - Byte offset

↓ spatial locality competitions
Larger block: miss rate ↓, #blocks ↓, miss rate ↑, pollution

miss penalty ↑ / cache size ↓ 容量 ↓ time ↑

11.3 Cache miss and hit

1. Miss categories

① Compnsory ② Capacity ③ Conflict

2. Read

① hit, normal ② miss: stall, fetch

3. Write

① hit (i) Write through (同步更新 cache, mem)

Basic CPL = 1, 10% store, 100 cycle of write

$$\Rightarrow \text{CPL} = 1 \times 0.9 + 0.1 \times 100 \approx 11$$

⇒ write buffer: continue CPU

(ii) Write back (只写 cache, 更新 dirty bit)

When kick out, if D=1, write mem

② miss (i) Write around 直接在 mem 改

(ii) Write allocate 换在 cache, 写

Lecture 12 Memory Hierarchy II

12.1 Cache Performance measurement

1. CPU Time = Program exe time + Mem stall cycles

(N ins) I-cache: $N \times \text{rate} \times \frac{\text{penalty}}{N}$

D-cache: $N \times \% \times \text{rate} \times \frac{\text{penalty}}{N}$

$$\text{CPL} = \text{CPL}_0 + \text{CPL}_2 + \text{CPL}_D$$

2. AMAT (avg mem access time)

$$\text{AMAT} = \text{Hit Time} + \text{Miss rate} \times \text{Miss penalty}$$

12.2 Set associate

1. Category / fully associate (any, Comparator per entries)

n-way set associate (at $\frac{\text{addr}}{n}$, n comparator)

2. Cal: #Set = #blocks / n-way

$$\text{index} = \log_2(\# \text{set})$$

V - Tag - Data
index

3. associativity ↑ miss ↓

收益变缓

4. Replacement Policy

① Direct: no choice ② Set: 先找无效

③ Policy: / LRU: Least recent used { 20% more periodical clear tag
Random

12.3 Multi-level cache 1GHz > 1ns

1. Categories

L1 For minimum hit time

L2 For low miss rate to avoid mem access

CPI = L1 hit + L1 miss, L2 hit + L1, L2 miss (global)

$$0.98 \times 1 + 0.015 \times 21 + 0.005 \times 421$$

12.4 Virtual Memory

1. Def: as a "cache" for disk

VM block is a page, miss is a page fault

VPN → 查表 → PPN, page offset 不变

2. Page fault penalty

Fully associative placement + Smart replacement

3. Page tables 数组

index | V | ref | D | PPN | VPN → PPN 桶

PPN = Page table [VPN]

#page entries = 2^{index}, index = $\frac{\text{VPN}}{\text{bit not Vaddr}}$

page table size = entry size × #entries

offset = $\log_2(\text{page size})$

4. Replacement and Write

① LRU: using ref bit, periodically set to 0, clear 0

② Disk: write back, dirty bit

12.5 Fast Translation using TLB

1. Def: fast cache for PTE

LRU tag

2. 组成 | V | ref | D | VPN | PPN

page size ⇒ page byte offset } fully associative

Vaddr >> offset = VPN

n-way associate:

index 选 set Vaddr >> offset 取低 $\log_2(n)$ bit

tag 判为 Vaddr >> offset 取高位 bit

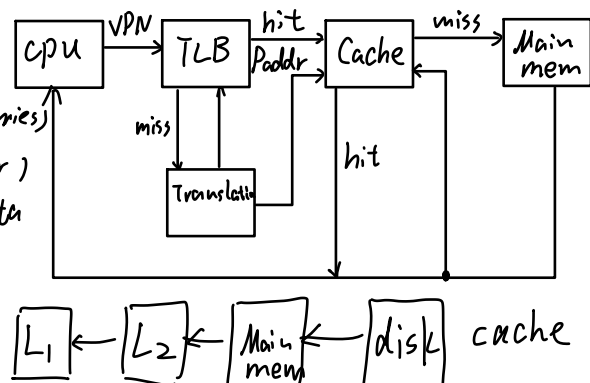
3. TLB miss

PTE

① If in memory: read from mem to TLB

retry

② If not (page fault): read from disk



Lecture 13 Memory Hierarchy III

13.1 Dependability Measures

MTTF mean time to failure

MTTR mean time to repair

MTBF mean time between failure = MTTF + MTTR

$$\text{availability} = \frac{\text{MTTF}}{\text{MTBF}} \begin{cases} 90\% & 365 \text{ d/yr} \\ 99\% & 3.65 \text{ d/yr} \\ 99.9\% & 526 \text{ min/yr} \end{cases} \begin{matrix} \text{repair} \\ \text{time} \end{matrix}$$

13.2 Hamming SEC code

1. Hamming distance

for a set of code, min hamming distance = d

能识别 d-1 个错误, 纠正 $\lfloor \frac{d-1}{2} \rfloor$ 个错误

min d = 3 → SEC single error correction

2. Encode SEC n bit data, k bit parity

$P_1 P_2 P_4 P_8$ as parity bit SEC: $2^k \geq n+k+1$

$P_1 P_2 d_1 P_4 d_2 d_3 d_4 P_8 d_5 d_6 d_7 d_8$ DEC=SEC+1

P_1 check right 1st 1 P_4 check right 3rd 1

P_2 check right 2nd 1 P_8 check right 4th 1

3. Decode SEC

run $P_1 P_2 P_4 P_8$ for sequence (xor each bit)

should be 0 0 0 0 e.g. if 0 1 0 1 ⇒ $\frac{1010}{10}$ 10 10 10 10

4. Another P_n for whole word

(1-based) 3. Shared Memory Multiprocessors

Make hamming distance = 4, SEC, DEC (double error)

$P_{1248} P_n$

✓ ✓ correct

✓ X P_n incorrect

X ✓ double bit error

X X single bit error

13.3 Virtual machine

1. VM

host isolate guests

2. VMM

maps virtual resources to physical resources

Lecture 14 Parallel Processors

14.1 Amdahl's law

1. Sequential part limits speed up

$$1P: F_{seq} + F_{par} = 1 (T_{old})$$

$$100P: T_{new} = F_{seq} + F_{par}/100$$

$$\text{speed up} = T_{old}/T_{new}$$

$$\text{Time} = \text{Scalar Time} + \text{Parallel Time}, \text{Ideal speed up} = \# \text{ of processors}$$

2. Strong/weak scaling

$$\text{Strong: } T(N, P) = T(1, P)/N$$

$$\text{Weak: } T(N_1, P_1) = T(N_2, P_2)$$

$$N_1 = kN_2, P_1 = kP_2$$

3. Load balancing

并行耗时取决于最慢处理器

20P, 400 add, 1 take 20, other take 380.

$$\text{Time} = \max(\frac{380}{19}t, 20t) = 20t$$

14.2 SISD, SIMD, MISD, MIMD, vector

Data Stream

Single

Multiple

Ins Single

SISD

SIMD

Stream Multiple

MISD

MIMD

1. SIMD and Vector

① Data level parallelism, 所有处理器相同操作

reduce ins control hardware

② Vector processors

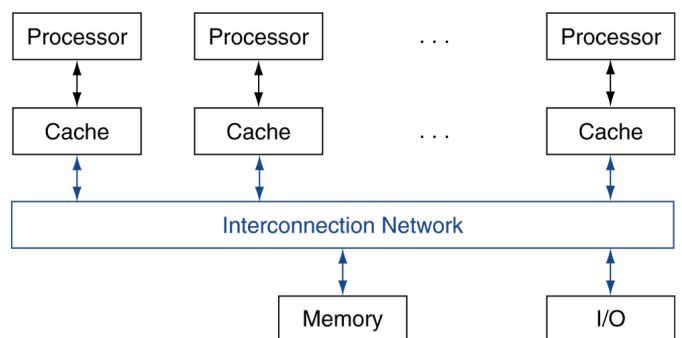
SIMD 实现方式, 1 ins → Ex n data

2. Multithreading (not MIMD, in 1 core)

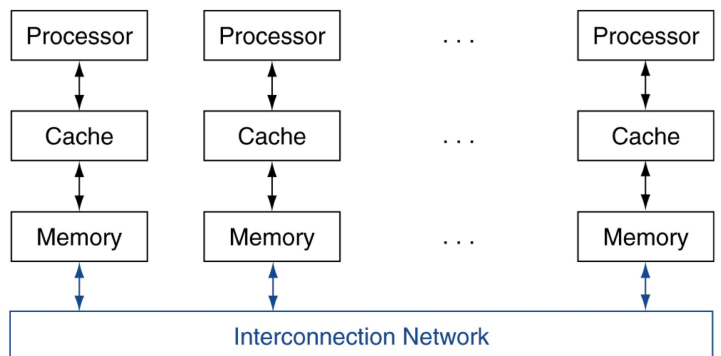
Coarse MT: switch on long stall (L2 cache miss)

Fine MT: Switch after each cycle

Simultaneous MT: schedule ins, 有空就执行, reg rename



4. Message Passing Multiprocessors



1. Classification

Static

Dynamic

ILP

VLIN

Superscalar

DLP

SIMD or Vector

Nvidia Multiprocessor